Module 4: Numerical Method for the Conventional MHD Solver

MHD_PINN_proj1 Physics Notes

Revised to connect every algorithmic piece to the underlying conservation laws

These notes explain how the conventional solver translates the ideal-MHD equations into a computable algorithm. The emphasis is not merely on listing the numerical ingredients. The goal is to show why a finite-volume method is the natural choice for conservation laws, how interface fluxes arise, why reconstruction and time integration are upgraded, and how each numerical decision relates to the underlying physics.

1 Symbol and variable table

Symbol	Name	Meaning
N_x, N_y	grid cell counts	Number of finite-volume cells in the two coordinate directions.
$\Delta x, \Delta y$	cell widths	Uniform spacing of the Cartesian grid.
Δt	time step	Explicit update step chosen from a Courant stability bound.
$\mathbf{U}_{i,j}$	cell-averaged conserved state	Average conserved state stored in cell (i, j).
$\mathbf{F}_{i+1/2,j}$	numerical flux through an x-face	Flux leaving or entering a cell across a vertical face.
$\mathbf{G}_{i,j+1/2}$	numerical flux through a y-face	Flux leaving or entering a cell across a horizontal face.
$a_{\max}$	maximum signal speed	Largest estimated wave speed relevant to a face flux.
$c_{f,x}, c_{f,y}$	fast magnetosonic speeds	Fastest characteristic speeds normal to the coordinate directions.
CFL	Courant number	Dimensionless safety factor controlling the explicit time step.
$\mathbf{U}_L, \mathbf{U}_R$	left and right interface states	Reconstructed states used to solve a face Riemann problem approximately.
σ_i	reconstructed slope	Cellwise slope used by piecewise-linear reconstruction.
minmod	minmod limiter	Nonlinear limiter used to reduce spurious oscillations near steep gradients.
$\mathcal{L}(\mathbf{U})$	spatial operator	Finite-volume flux-difference operator appearing in the semidiscrete system.
$\nabla \cdot \mathbf{B}$	magnetic divergence	Diagnostic quantity that should remain small in a valid MHD solution.

2 Why the project begins with a conventional solver

Before a physics-informed neural network can be judged honestly, the project needs a direct numerical method that solves the intended equations in a transparent way. The conventional solver supplies:

- a baseline for what the equations predict,
- physically meaningful diagnostics,
- a way to distinguish modeling error from neural-network training error,
- a source of reference solutions for later comparison.

Scientific reason. A PINN without a conventional baseline is hard to interpret. If the network prediction looks plausible, there is no trustworthy anchor. If it looks implausible, there is no clear way to separate physics error from optimizer failure.

3 Why a finite-volume method is the natural language of conservation laws

3.1 Start from the conservative PDE

The ideal-MHD system is written in conservation-law form:

$$\frac{\partial \mathbf{U}}{\partial t} + \frac{\partial \mathbf{F}(\mathbf{U})}{\partial x} + \frac{\partial \mathbf{G}(\mathbf{U})}{\partial y} = 0. \tag{1}$$

This equation says that the time rate of change of a conserved quantity inside a region is determined by the net flux through the region's boundary.

3.2 Integrate over one control volume

Take one grid cell:

$$V_{ij} = [x_{i-1/2}, x_{i+1/2}] \times [y_{j-1/2}, y_{j+1/2}]. \tag{2}$$

Integrate (**??**) over that cell:

$$\int_{V_{ij}} \frac{\partial \mathbf{U}}{\partial t}\, dA + \int_{V_{ij}} \frac{\partial \mathbf{F}}{\partial x}\, dA + \int_{V_{ij}} \frac{\partial \mathbf{G}}{\partial y}\, dA = 0. \tag{3}$$

Bring the time derivative outside the integral:

$$\frac{\mathrm{d}}{\mathrm{d}t} \int_{V_{ij}} \mathbf{U}\, dA + \int_{V_{ij}} \frac{\partial \mathbf{F}}{\partial x}\, dA + \int_{V_{ij}} \frac{\partial \mathbf{G}}{\partial y}\, dA = 0. \tag{4}$$

3.3 Convert flux-divergence terms to face terms

Because the grid is Cartesian,

$$\int_{V_{ij}} \frac{\partial \mathbf{F}}{\partial x}\, dA = \int_{y_{j-1/2}}^{y_{j+1/2}} \Big[\mathbf{F}(x_{i+1/2}, y) - \mathbf{F}(x_{i-1/2}, y)\Big]\, dy. \tag{5}$$

Likewise,

$$\int_{V_{ij}} \frac{\partial \mathbf{G}}{\partial y}\, dA = \int_{x_{i-1/2}}^{x_{i+1/2}} \Big[\mathbf{G}(x, y_{j+1/2}) - \mathbf{G}(x, y_{j-1/2})\Big]\, dx. \tag{6}$$

Define the cell average

$$\mathbf{U}_{ij} = \frac{1}{\Delta x \Delta y} \int_{V_{ij}} \mathbf{U}\, dA. \tag{7}$$

Then the exact finite-volume balance law becomes

$$\frac{\mathrm{d}\mathbf{U}_{ij}}{\mathrm{d}t} = -\frac{1}{\Delta x}\left(\mathbf{F}_{i+1/2,j} - \mathbf{F}_{i-1/2,j}\right) - \frac{1}{\Delta y}\left(\mathbf{G}_{i,j+1/2} - \mathbf{G}_{i,j-1/2}\right). \tag{8}$$

3.4 What this means physically

Equation (**??**) has a simple interpretation:

- if more mass leaves a cell than enters, the cell mass decreases,
- if more momentum leaves than enters, the cell momentum decreases,
- if more total energy leaves than enters, the cell energy decreases.

The same face-flux logic applies to each row of the conserved vector.

4 Why a numerical flux is needed

At a cell face, the left and right neighboring cells generally do not agree. So the physical face flux is ambiguous: which state should be used? That is the local Riemann-problem idea. The project begins with a robust approximate answer: the Rusanov flux.

4.1 Rusanov flux

At a face with left and right states $\mathbf{U}_L$ and $\mathbf{U}_R$,

$$\mathbf{F}^{\mathrm{num}} = \frac{1}{2}\left[\mathbf{F}(\mathbf{U}_L) + \mathbf{F}(\mathbf{U}_R)\right] - \frac{1}{2} a_{\max}(\mathbf{U}_R - \mathbf{U}_L). \tag{9}$$

The same structure applies in the y direction with $\mathbf{G}$.

4.2 Interpretation of the two terms

The first term is a centered average of the physical fluxes. If the left and right states are similar, this part dominates. The second term is dissipative. It damps sharp jumps in proportion to a wave-speed estimate.

That second term is why Rusanov is robust:

- if a discontinuity tries to create unstable numerical behavior, the dissipative term opposes it,
- the dissipation is stronger when the characteristic speeds are larger,
- the price paid is extra numerical diffusion.

4.3 Why the signal speed must be large enough

The speed $a_{\max}$ must dominate the fastest information that can cross the face. In MHD, a safe choice is the fast magnetosonic speed. For propagation normal to the x direction,

$$c_{f,x}^2 = \frac{1}{2}\left[a^2 + b^2 + \sqrt{(a^2+b^2)^2 - 4a^2 b_x^2}\right], \tag{10}$$

where

$$a^2 = \frac{\gamma p}{\rho}, \qquad b^2 = \frac{B_x^2 + B_y^2 + B_z^2}{\rho}, \qquad b_x^2 = \frac{B_x^2}{\rho}. \tag{11}$$

Then a common estimate is

$$a_{\max,x} = |u_x| + c_{f,x}. \tag{12}$$

The same logic produces $a_{\max,y} = |u_y| + c_{f,y}$.

4.4 Why fast magnetosonic speed is the conservative choice

MHD has several wave families. The fast mode is the broadest characteristic envelope, so using it is a conservative stability decision. It may add more diffusion than necessary in some places, but it protects the method from underestimating how fast information can move.

5 Why the time step must satisfy a CFL condition

5.1 Physical intuition

An explicit method updates the solution using only information from the current time level. Therefore one time step cannot be allowed to move information across too many cells. Otherwise the update would be trying to propagate signals farther than the stencil can represent consistently.

5.2 CFL form

A common stability restriction is

$$\Delta t = \mathrm{CFL} \cdot \min\left(\frac{\Delta x}{a_{\max,x}}, \frac{\Delta y}{a_{\max,y}}\right). \tag{13}$$

If the characteristic speeds grow, the allowed time step shrinks. This is not arbitrary. It reflects the physical fact that faster wave motion must be resolved by smaller explicit time increments.

6 Why piecewise-linear reconstruction is needed

6.1 What goes wrong with piecewise-constant data

If each cell is treated as constant, then every face sees left and right states that jump sharply from one average to the next. That is robust, but it smears smooth features. The Alfvén benchmark makes this obvious because a smooth sinusoid becomes too diffused.

6.2 Piecewise-linear reconstruction

In one dimension, approximate a quantity q inside cell i by

$$q(x) \approx q_i + \sigma_i(x - x_i), \tag{14}$$

where σ_i is a slope estimate. Then the left and right edge states are

$$q_i^L = q_i - \frac{1}{2}\sigma_i \Delta x, \qquad q_i^R = q_i + \frac{1}{2}\sigma_i \Delta x. \tag{15}$$

This gives the numerical flux a more realistic estimate of what the solution looks like at the interface.

6.3 Why a limiter is needed

A naive centered slope may create new extrema or oscillations near steep gradients. The minmod limiter avoids that by comparing one-sided differences. If

$$a = \frac{q_i - q_{i-1}}{\Delta x}, \qquad b = \frac{q_{i+1} - q_i}{\Delta x}, \tag{16}$$

then

$$\operatorname{minmod}(a, b) = \begin{cases} \operatorname{sign}(a)\min(|a|, |b|), & ab > 0, \\ 0, & ab \le 0. \end{cases} \tag{17}$$

So:

- in a smooth monotone region, the slope is retained in a cautious way,
- near a local extremum or sign change, the slope is suppressed,
- the method becomes less oscillatory and more robust.

7 Why Runge–Kutta 2 is used for time integration

The semidiscrete system (**??**) can be written abstractly as

$$\frac{\mathrm{d}\mathbf{U}}{\mathrm{d}t} = \mathcal{L}(\mathbf{U}), \tag{18}$$

where $\mathcal{L}$ is the flux-difference operator. A first-order Euler step is

$$\mathbf{U}^{n+1} = \mathbf{U}^n + \Delta t\, \mathcal{L}(\mathbf{U}^n), \tag{19}$$

but that is only first-order accurate in time.

The two-stage Runge–Kutta update is

$$\mathbf{U}^{(1)} = \mathbf{U}^n + \Delta t\, \mathcal{L}(\mathbf{U}^n), \tag{20}$$

$$\mathbf{U}^{n+1} = \frac{1}{2}\mathbf{U}^n + \frac{1}{2}\left[\mathbf{U}^{(1)} + \Delta t\, \mathcal{L}(\mathbf{U}^{(1)})\right]. \tag{21}$$

This gives a better time approximation for smooth evolution. Since PLM is already improving the spatial approximation, pairing it with a second-order time integrator makes the overall method more balanced.

8 Why boundary conditions matter physically

For the Alfvén verification problem, periodic boundaries are used in x. That means:

$$\mathbf{U}(x_{\min}, y, t) = \mathbf{U}(x_{\max}, y, t). \tag{22}$$

Physically, this corresponds to a repeating domain. Numerically, it allows a traveling sinusoid to leave one side and re-enter the other without artificial reflection.

In the code, periodicity is often implemented through ghost cells copied from the opposite side. That sounds like a code detail, but it is actually a direct numerical representation of the domain topology.

9 Why magnetic divergence is monitored as a diagnostic

The exact equations require

$$\nabla \cdot \mathbf{B} = 0. \tag{23}$$

In two dimensions,

$$\nabla \cdot \mathbf{B} = \frac{\partial B_x}{\partial x} + \frac{\partial B_y}{\partial y}. \tag{24}$$

A finite-volume code that does not use a specialized divergence-control method will generally not preserve this quantity exactly. Monitoring it is therefore essential. If the diagnostic grows too large, the numerical solution may develop unphysical magnetic forces.

10 Why positivity floors are included

Density and gas pressure must remain physically meaningful:

$$\rho > 0, \qquad p > 0. \tag{25}$$

Near strong gradients or under-resolved features, an explicit scheme can sometimes produce small negative values due to numerical error. A positivity floor is a pragmatic safeguard:

- it prevents the update from entering a mathematically invalid state,
- it keeps wave-speed calculations and flux functions defined,
- it makes debugging easier during development.

It is not the final ideal cure, but it is an honest and useful development tool.

11 How one full solver step fits together

Putting all the pieces together, one upgraded solver step looks like this:

1. start from cell-averaged conserved variables,
2. recover primitive variables where needed,
3. reconstruct left and right interface states with PLM and minmod limiting,
4. compute numerical fluxes with the Rusanov formula,

5. assemble the finite-volume spatial operator $\mathcal{L}$,
6. advance with Runge–Kutta 2,
7. optionally apply positivity floors,
8. evaluate diagnostics such as L^2 error and $\max|\nabla \cdot \mathbf{B}|$.

Each of those steps traces back to a physical or mathematical requirement: conservation, characteristic propagation, stability, accuracy, or admissibility.

Bottom line. The conventional solver is not a random collection of MATLAB routines. It is a structured translation of the conservation-law form of ideal MHD into a sequence of operations that preserve flux balance, approximate wave propagation, control numerical instability, and expose meaningful diagnostics.

12 What Module 4 accomplishes

Module 4 explains why the conventional solver is built the way it is:

- finite volume because the equations are conservation laws,
- face fluxes because conserved quantities change by crossing cell boundaries,
- Rusanov because it is robust and easy to implement,
- PLM plus minmod because the solver must improve smooth-wave resolution without becoming oscillatory,
- Runge–Kutta 2 because the time integration should match the upgraded spatial accuracy,
- diagnostics and floors because development must remain physically honest.

That makes the numerical method understandable as physics translated into an algorithm, not merely as a list of coding choices.